POOGym

Guia prática de desenvolvimento — Fase 1

Prazo indicado no enunciado: 25 de outubro de 2026 | Peso: 3 valores (de 7)

Objetivo

Desenvolver em Java uma aplicação de gestão de ginásio para gerir sócios, instrutores, atividades e marcações, aplicando encapsulamento, herança e coleções. Este documento resume os requisitos e propõe uma ordem de trabalho e uma divisão entre dois colegas. A proposta de organização é uma sugestão; os requisitos funcionais abaixo vêm do enunciado.

1. Entregáveis da Fase 1

- Código-fonte de todas as classes Java, sem erros de compilação.
- Diagrama de classes UML em PDF, correspondente ao código entregue.
- Relatório em PDF.
- ZIP com o Javadoc gerado a partir dos comentários.
- Dados de demonstração: pelo menos 6 sócios (3 de cada tipo), 8 atividades e 6 marcações.

Na Fase 1, os dados de demonstração podem ser criados diretamente no código. Não é necessário usar ficheiros.

2. Requisitos do domínio

Sócios

- Dados comuns: nome, número de sócio único e e-mail.
- Sócio individual: ano de nascimento; desconto de 10% para menores de 18 anos e 15% para pessoas com 65 anos ou mais.
- Sócio empresa: nome da empresa parceira e NIF; desconto fixo de 20% em todas as marcações.

Instrutor

Nome, género e formação (texto com a lista de cursos e formações).

Atividades

- Aula de grupo: nome, instrutor, data/hora de início, duração em minutos, modalidade e capacidade máxima. Modalidades: Yoga, Spinning, Zumba e CrossFit.
- Sessão de personal training: nome, instrutor, data/hora de início, duração em minutos e nível: Iniciação, Intermédio ou Avançado.

Atividade	Custo base
Yoga	8,00 €
Spinning	9,00 €
Zumba	8,50 €

Atividade	Custo base
CrossFit	12,00 €
PT — Iniciação	18,00 €
PT — Intermédio	22,00 €
PT — Avançado	28,00 €

O enunciado define estes custos base por atividade; não indica que a duração altere o preço.

3. As 10 funcionalidades obrigatórias

1. Registar sócio — Pedir dados e criar sócio individual ou empresa.
2. Registar instrutor — Pedir os dados e criar um instrutor/personal trainer.
3. Registar atividade — Criar aula de grupo ou sessão de personal training.
4. Listar sócios — Mostrar todos os sócios com os respetivos dados.
5. Listar atividades — Mostrar todas as atividades com os respetivos dados.
6. Criar marcação — Associar sócio a atividade, registar data/hora e calcular custo com desconto.
7. Listar marcações — Mostrar todas as marcações, os custos e o total faturado.
8. Resumo de atividades — Contar sócios individuais/empresa e aulas de grupo/sessões de PT.
9. Mais popular — Identificar a atividade com mais marcações.
10. Valores Sócio — Pedir o número de sócio e calcular o total gasto e o total poupado.

4. Cálculos financeiros

Poupança = preço base × desconto / 100
Preço final = preço base − poupança

Exemplo: CrossFit custa 12,00 €. Com desconto de 15%, a poupança é 1,80 € e o preço final é 10,20 €.

- Total faturado: soma dos preços finais de todas as marcações.
- Total gasto por sócio: soma dos preços finais das marcações desse sócio.
- Total poupado por sócio: soma da diferença entre o preço base e o preço final nas suas marcações.
- Sem marcações, os totais do sócio devem ser 0 €. Para um número de sócio inexistente, apresentar mensagem apropriada.

5. Estrutura de classes sugerida

Classe	Responsabilidade
Socio	Dados comuns: nome, número e e-mail.
SocioIndividual	Ano de nascimento e desconto por idade.
SocioEmpresa	Nome da empresa, NIF e desconto de 20%.
Instrutor	Nome, género e formação.
Atividade	Dados comuns: nome, instrutor, início e duração.
AulaGrupo	Modalidade e capacidade máxima.
SessaoPT	Nível de treino.

Classe	Responsabilidade
Marcacao	Sócio, atividade, data/hora e valores financeiros.
GestorGinasio (sugestão)	Coleções e operações de registo, pesquisa, listagem e estatísticas.
Main	Arranque, dados de demonstração e menu.

A estrutura é uma proposta. Na Fase 1, não usar classes abstratas nem polimorfismo. Usar herança simples e manter os atributos encapsulados.

6. Coleções e boas práticas

Coleções possíveis, se já tiverem sido abordadas nas aulas:

ArrayList<Socio> socios;
ArrayList<Instrutor> instrutores;
ArrayList<Atividade> atividades;
ArrayList<Marcacao> marcacoes;

- Manter os atributos normalmente private e expor apenas métodos necessários.
- Cada classe deve ter uma responsabilidade clara; não concentrar toda a lógica no main.
- Evitar código redundante, variáveis/métodos públicos desnecessários e uso injustificado de static.
- Comentar classes e métodos públicos com Javadoc.
- O enunciado avisa que instanceof e class.getName() serão penalizados.

7. Plano de trabalho até à entrega

Data	Objetivo
17 out.	Analisar requisitos, definir classes e assinaturas, desenhar UML inicial e criar o projeto.
18–19 out.	Implementar classes do domínio, construtores, encapsulamento, descontos e marcação.
20–21 out.	Implementar gestor, menu, registos, listagens e as dez funcionalidades.
22 out.	Criar os dados de demonstração e integrar os módulos.
23 out.	Testar requisitos, casos inválidos, cálculos, estatísticas e corrigir erros.
24 out.	Finalizar UML, Javadoc e relatório; organizar os ficheiros de entrega.
25 out.	Compilar, executar um teste completo e submeter no InforEstudante.

Este calendário é uma proposta de organização, não um calendário oficial do docente.

8. Divisão entre dois colegas

Pessoa A — modelo e regras de negócio

- Implementar Socio, SocioIndividual e SocioEmpresa.
- Implementar Instrutor, Atividade, AulaGrupo e SessaoPT.
- Implementar Marcacao e cálculos de preço final/poupança.
- Testar descontos e dados dos objetos; preparar parte inicial do UML.

Pessoa B — gestão e funcionalidades

- Implementar GestorGinasio e as quatro coleções.
- Implementar operações de registo, pesquisa e listagem.
- Implementar resumo por categorias, atividade mais popular e valores por sócio.
- Implementar Main/menu e testar a interação.

Em conjunto

- Combinar nomes de classes, atributos, construtores e assinaturas dos métodos antes de programar em paralelo.
- Integrar e compilar diariamente; resolver incompatibilidades cedo.
- Rever UML final, relatório, Javadoc e dados de demonstração.
- Ambos devem compreender todo o código para a defesa individual.

Exemplos de assinaturas a combinar

```
void registarSocio(Socio socio);
void registarInstrutor(Instrutor instrutor);
void registarAtividade(Atividade atividade);
void criarMarcacao(Marcacao marcacao);
```

São exemplos de interfaces possíveis, não assinaturas obrigatórias.

9. Checklist de testes

- ☐ Registar sócio individual e empresa; rejeitar número duplicado.

- ☐ Registar instrutor e atividades; listar os dados corretamente.
- ☐ Verificar os descontos de 10%, 15% e 20%.
- ☐ Criar marcações e confirmar preço final, poupança e faturação.
- ☐ Confirmar contagens por categoria e atividade mais popular.
- ☐ Consultar sócio com várias marcações e sócio sem marcações.
- ☐ Testar número de sócio inexistente, coleções vazias e empates.
- ☐ Executar todas as opções do menu sem erros.
- ☐ Confirmar seis sócios (três de cada tipo), oito atividades e seis marcações.
- ☐ Compilar todos os .java, gerar Javadoc e verificar UML e relatório.

10. Preparação para a defesa

Ambos devem conseguir explicar herança, encapsulamento, coleções, cálculo de descontos, estatísticas e responsabilidades das classes. Treinem alterações pequenas no código. A defesa é individual e pode incluir perguntas sobre o projeto e desafios de alteração de funcionalidades.

11. O que deixar para a Fase 2

- Leitura e escrita de ficheiros de texto e objetos.
- Classes abstratas e polimorfismo.
- Carregamento e persistência dos dados.
- Funcionalidades adicionais específicas da Fase 2.

Nota sobre IA: o enunciado recomenda utilização crítica para esclarecimentos e debugging, e não prompts destinados a gerar a solução completa. Ambos devem compreender e conseguir defender o trabalho.